

Examen d'entraînement LEPL1110 — Éléments Finis

Claude code

Mai 2026

Durée : 3 heures. Aucun document autorisé. Calculatrice non autorisée.

Les exercices sont indépendants et peuvent être traités dans n'importe quel ordre.

Exercice 1 — Équation de diffusion de la chaleur en 1D

Une barre métallique de longueur $L = 1$, de diffusivité thermique $\kappa > 0$, est thermiquement isolée à ses deux extrémités. La température $u(x, t)$ est solution de l'équation de diffusion :

$$\partial_t u - \kappa \partial_{xx} u = 0, \quad x \in (0, 1), \quad t > 0,$$

avec les **conditions aux limites de Neumann homogènes** :

$$\partial_x u(0, t) = 0, \quad \partial_x u(1, t) = 0,$$

et la **condition initiale** $u(x, 0) = u_0(x)$.

1.1 — Formulation faible en espace

Multiplier l'équation de diffusion par une fonction test $v \in H^1(0, 1)$ et intégrer sur $(0, 1)$. En utilisant une intégration par parties, montrer que la formulation variationnelle s'écrit :

$$\int_0^1 \partial_t u v \, dx + \kappa \int_0^1 \partial_x u \partial_x v \, dx = 0, \quad \forall v \in H^1(0, 1).$$

Expliquer pourquoi les conditions aux limites de Neumann n'apparaissent pas explicitement dans cette formulation.

1.2 — Semi-discrétisation en espace

On cherche une approximation $u_h(x, t) = \sum_{j=0}^N u_j(t) \varphi_j(x)$, où $\{\varphi_j\}$ est une base d'éléments finis P_1 sur un maillage de N éléments uniformes.

En choisissant $v = \varphi_i$, montrer que le vecteur $\mathbf{u}(t)$ vérifie :

$$M \dot{\mathbf{u}}(t) + \kappa K \mathbf{u}(t) = \mathbf{0},$$

où $M_{ij} = \int_0^1 \varphi_j \varphi_i \, dx$ et $K_{ij} = \int_0^1 \partial_x \varphi_j \partial_x \varphi_i \, dx$.

1.3 — Matrices élémentaires et assemblage ($N = 2$)

On discrétise avec $N = 2$ éléments P_1 uniformes de longueur $h = 1/2$. Les nœuds sont en $x_0 = 0$, $x_1 = 1/2$, $x_2 = 1$.

(a) Calculer la matrice de rigidité élémentaire $K^{(e)}$ et la matrice de masse élémentaire $M^{(e)}$ pour un élément de longueur h .

Rappel : fonctions de forme $\varphi_0 = (x_{e+1} - x)/h$, $\varphi_1 = (x - x_e)/h$.

(b) Assembler les matrices globales M et K de taille 3×3 .

(c) Vérifier que $K \mathbf{1} = \mathbf{0}$ où $\mathbf{1} = (1, 1, 1)^\top$. Donner une interprétation physique.

1.4 — Schéma en temps (schéma θ)

(a) Écrire le **schéma** θ pour le système $M\dot{\mathbf{u}} + \kappa K\mathbf{u} = \mathbf{0}$. Pour quelles valeurs de θ obtient-on l'Euler explicite, l'Euler implicite et Crank–Nicolson ?

(b) Montrer que $\frac{d}{dt}\|u(t)\|_{L^2}^2 \leq 0$ pour le problème continu. Pour quelle valeur de θ le schéma est-il **inconditionnellement** L^2 -stable ?

(c) Pour $\theta = 0$, exprimer la **condition de stabilité** sur Δt en fonction de la plus grande valeur propre $\lambda_{\max}$ du problème généralisé $K\phi = \lambda M\phi$.

1.5 — Analyse modale et solution exacte

(a) Trouver les fonctions propres $\phi_n(x)$ et valeurs propres λ_n de $-\phi_n'' = \lambda_n\phi_n$ sur $(0, 1)$ avec $\phi_n'(0) = \phi_n'(1) = 0$.

(b) Pour $u_0(x) = \cos(\pi x)$, écrire la solution exacte $u(x, t)$.

(c) Quel est le temps caractéristique τ du mode dominant non constant ? Vers quelle valeur la température se stabilise-t-elle ?

Exercice 2 — Le jeu des erreurs

Le code ci-dessous implémente l'équation de diffusion 1D avec N éléments P_1 uniformes et une quadrature de Gauss–Legendre à 2 points. Il est censé retourner les matrices M et K .

Ce code contient volontairement un maximum de sept erreurs.

Repérez ces erreurs à l'aide des numéros ① à ⑦ et fournissez un bref commentaire pour chacune.

```
1 import numpy as np
2
3 def assemble_heat_1d(N, kappa):
4     x = np.linspace(0.0, 1.0, N)                # a annoter si errone
5
6     M = np.zeros((N + 1, N + 1))
7     K = np.zeros((N + 1, N + 1))
8
9     xi_q = [-1.0 / np.sqrt(3.0), 1.0 / np.sqrt(3.0)]
10    w_q = [1.0, 1.0]
11
12    dN_dxi = [-0.5, 0.5]
13
14    for e in range(N):
15        x0 = x[e]
16        x1 = x[e + 1]
17        h = x1 - x0
18        Jac = h                                # a annoter si errone
19        dxi_dx = 1.0 / Jac
20
21        dof = [e, e + 1]
22
23        for q in range(2):
24            xi = xi_q[q]
25            w = w_q[q]
26
27            Nq = [0.0, 0.0]
28            Nq[0] = 0.5 * (1.0 + xi)            # a annoter si errone
29            Nq[1] = 0.5 * (1.0 - xi)            # a annoter si errone
30
31            dN_dx = [0.0, 0.0]
32            for i in range(2):
33                dN_dx[i] = dN_dxi[i] / dxi_dx    # a annoter si
34                errone
35
36            for i in range(2):
37                I = dof[i]
38                for j in range(2):
39                    J = dof[j]                    # a annoter si errone
40
41                    M[I, J] += Nq[i] * Nq[j] * w * Jac
42                    K[I, J] += kappa * dN_dx[i] * dN_dx[j] * Jac    #
43                    a annoter si errone
44
45    return K, M                                # a annoter si errone
```

Liste des erreurs (à compléter) :

- ① _____
- ② _____
- ③ _____
- ④ _____
- ⑤ _____
- ⑥ _____
- ⑦ _____